

Lab Assignment: Data Breach Analysis & Database Exploration

Learning Objectives

By the end of this lab, you will be able to:

- **Assess the impact of a data breach** by analyzing a real dump (volume of personal data exposed, sensitivity, GDPR implications).
- **Set up a local relational database** from a raw SQL dump.
- **Connect Python to MariaDB/MySQL** using `SQLAlchemy` and load query results into `pandas` DataFrames.
- **Explore an unknown database schema** to map out tables, columns, and relationships independently.
- **Execute analytical SQL queries** (filtering, date manipulation, aggregations, joins) to generate synthetic metrics, CSV exports, and an exposure synthesis.

Step 0 — Setup and Local Database Import (Autonomous)

You may choose your own environment settings (database name, credentials, port, etc.). Keep track of these settings, as your Python scripts will need to connect to this database.

1. Obtain the project SQL dump file (`.sql`).
2. Set up a local MariaDB/MySQL instance using your preferred method (Docker container, native system service, local server stack, etc.).
3. Import the SQL dump into your database.
4. Verify that the import succeeded: the database should contain several hundred tables. Write a query to confirm the total table count.

Step 1 — Python Connection & Schema Discovery

Constraint: All analysis must be executed through Python using `SQLAlchemy` and `pandas`.

```
import os
import pandas as pd
from sqlalchemy import create_engine, inspect

# Adapt the connection string to your configuration (PyMySQL driver)
# Best practice: Retrieve credentials from environment variables
DB_URL = os.getenv("DATABASE_URL",
```

```
"mysql+pymysql://USER:PASSWORD@127.0.0.1:3306/DB_NAME")
engine = create_engine(DB_URL)

# Inspect available tables
inspector = inspect(engine)
print(inspector.get_table_names())

# Execute SQL query into a pandas DataFrame
df = pd.read_sql("SELECT COUNT(*) AS total_tables FROM
information_schema.tables WHERE table_schema = DATABASE()", engine)
print(df)
```

Methodology Tip: The `information_schema` database (`tables`, `columns`) and `inspect(engine)` from SQLAlchemy are key tools for mapping an unknown schema. Look for target tables using pattern matching (`LIKE '%...%'`).

Tasks

- **1.1 Table Discovery:** Write a script that connects to the database and lists all tables alongside their estimated row counts. Identify the core tables (e.g., users, profiles, messages).
- **1.2 Schema Inspection:** Select 3 to 4 key tables, display their column structures and data types, and inspect a sample of rows. Document your findings for each table.

Step 2 — User Account Analysis

Identify the table(s) storing user account details and perform the following analysis:

- **2.1 Volumetrics:** Determine the total number of accounts, how many are active, and how many have verified email addresses (identify the relevant status columns).
- **2.2 Registration Dynamics:** Calculate monthly user registrations.
 - *Note:* Dates may be stored as Unix timestamps. Convert them using SQL (`FROM_UNIXTIME(...)`) or within `pandas` (`pd.to_datetime(...)`).
- **2.3 Top Email Domains:** Extract and list the top 10 most frequent email domains.

Step 3 — Member Profiles & Relational Joins

User data is split across multiple tables linked by foreign keys. Reconstruct these relationships to map account credentials to public member profiles.

- **3.1 Join Verification:** Establish the join between user accounts and public profiles. Validate the output on sample records to ensure correct mapping (e.g., distinguishing account IDs from profile IDs).
- **3.2 Demographics:** Analyze member breakdown by gender.

- **3.3 Age Distribution:** Calculate member ages from birth dates and group them into age brackets (`<18` , `18-25` , `26-40` , `41-60` , `60+`). Handle missing or invalid values properly.
- **3.4 Profile Popularity:** Identify the top 10 most viewed public profiles.

Step 4 — Messaging System Analysis

Identify the tables governing private messaging (conversations and individual messages).

- **4.1 General Volumetrics:** Calculate the total number of conversations and total messages sent.
- **4.2 Activity Trends:** Determine the monthly distribution of sent messages.
- **4.3 Active Conversations:** Identify the top 10 most active conversations (by message volume), including their titles.
- **4.4 Active Users:** Identify the top 10 most active users in the messaging system.

Step 5 — Dashboard & Data Export

- **5.1 Executive Summary:** Write a script that produces a consolidated summary table (`pandas` `DataFrame`) containing key KPIs: total accounts, active accounts, verified emails, total profiles, total conversations, total messages, and the earliest/latest registration dates.
- **5.2 CSV Export:** Export at least one aggregated dataset (e.g., monthly registrations) to a `.csv` file using `DataFrame.to_csv(...)` . Ensure the file formats correctly in standard spreadsheet software.
- **5.3 Synthesis Report:** Write a concise analysis (5–10 lines) addressing:
 - Peak registration periods and overall user trends.
 - Typical community demographic profile.
 - Major GDPR and privacy vulnerabilities identified during your inspection (e.g., presence of sensitive data, timestamp anomalies, ghost accounts, security posture).

Note: Do not include raw personal data or unmasked credentials in your final submission (anonymize email addresses and full names).

Submission Deliverables

Submit a single zip archive containing:

1. **Python Scripts (`.py`):** Modular, well-commented code split logically by step. Use environment variables for connection parameters instead of hardcoding credentials.
2. **Exported Files:** Generated `.csv` output file(s).

3. **Technical Report (Markdown or PDF):** Documentation covering your methodology, structural discovery process, executed queries, numerical results, and analytical observations.
4. **Environment File:** A `requirements.txt` listing project dependencies (`sqlalchemy` , `pandas` , `pymysql`).

Optional Extensions (Bonus)

- **Full-Text Search Benchmarking:** Compare `LIKE '%keyword%'` queries against `FULLTEXT` indexing (`MATCH ... AGAINST`) on profile descriptions in terms of performance and relevance.
- **Data Visualization:** Plot monthly user registration trends using `matplotlib` or `seaborn` .
- **Data Quality Audit:** Detect anomalous data entries (aberrant birth dates, unverified dormant accounts, duplicate email addresses).
- **Security Assessment (Read-Only):** Evaluate how sensitive fields and passwords are stored (hash algorithm signatures, hash length, salt presence) and comment on password storage robustness without performing any cracking or attacks.